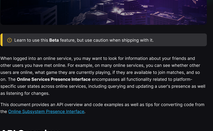
Join our community, grow your knowledge and learn from others!
Sign In
Don't have an Epic Games account? Sign up

Online Services Presence Interface

Access the presence and joinability status of friends and followers.



Learn to use this **Beta** feature, but use caution when shipping with it.

When logged into an online service, you may want to look for information about your friends and other users you have met online. For example, on many online services, you can see whether other users are online, what game they are currently playing, if they are available to join matches, and so on. The **Online Services Presence Interface** encompasses all functionality related to platform-specific user states across online services, including querying and updating a user's presence as well as listening for changes.

This document provides an API overview and code examples as well as tips for converting code from the Online Subsystem Presence Interface.

API Overview

Functions

The following table provides a high-level description of the functions included in the Presence Interface.

Function	Description
Query	
QueryPresence	Fetch the presence of the user with the supplied `TargetUserId`.
BatchQueryPresence	Fetch the presence for every user in the supplied list of `TargetUserIds`.
Get	
GetCachedPresence	Retrieve the presence of the user with the supplied `TargetUserId`, cached in the interface.
Update	
UpdatePresence	Update the presence of the user.
PartialUpdatePresence	Update the presence of the user with only the specified presence settings.
Event Listening	
OnPresenceUpdated	Event will trigger as a result of updates to a user's presence.

Enumeration Classes

The Presence Interface defines three enumeration classes that represent a user's status (`EUserPresenceStatus`), joinability (`EUserPresenceJoinability`), and game status (`EUserPresenceGameStatus`). These enumeration classes represent three primary members of the `FUserPresence` struct. For more information, refer to the Primary Struct section of this page.

EUserPresenceStatus

Enumerator	Description
`Offline`	User is offline.
`Online`	User is online.
`Away`	User is away.
`ExtendedAway`	User has been away for at least two hours (may be platform dependent).
`DoNotDisturb`	User does not want to be disturbed.
`Unknown`	Default user presence status.

EUserPresenceJoinability

Enumerator	Description
`Public`	Anyone can discover and join this session.
`FriendsOnly`	Anyone trying to join must be a friend of a lobby member.
`InviteOnly`	Anyone trying to join must be invited first.
`Private`	User is not currently accepting invitations.
`Unknown`	Default user presence joinability status.

EUserPresenceGameStatus

Enumerator	Description
`PlayingThisGame`	User is playing the same game as you.
`PlayingOtherGame`	User is playing a different game than you.
`Unknown`	Default user presence game status.

Primary Struct

FUserPresence

The `FUserPresence` struct is the primary object in the Presence Interface and consists of all necessary information pertaining to a user's presence.

Member	Type	Description
`AccountId`	`FAccountId`	User whose presence this is.
`Status`	`EUserPresenceStatus`	User presence state. (Default value is `EUserPresenceStatus::Unknown`.)
`Joinability`	`EUserPresenceJoinability`	User session state. (Default value is `EUserPresenceJoinability::Unknown`.)
`GameStatus`	`EUserPresenceGameStatus`	User game state. (Default value is `EUserPresenceGameStatus::Unknown`.)
`StatusString`	`FString`	String representation of user presence state.
`RichPresenceString`	`FString`	Game-defined representation of the current game state.
`Properties`	`FPresenceProperties`	Session keys.

The type `FPresenceProperties` is a typedef for `TMapPtrKeyFuncs<FPresenceVariant>`, where `FPresenceVariant` is an `FString`.

Examples

We now provide an example demonstrating `UpdatePresence`, `QueryPresence`, and `GetPresence` (`QuickUpdate`) updates that presence with the default platform services, then `QuickUpdate` queries the presence of `QuickUser`, after it has been updated. If the query successfully returns, then `QuickUpdate` retrieves the presence of `QuickUser`.

Code

UserAkPresence.cpp

```
// Copyright Epic Games, Inc. All Rights Reserved.
IMPLEMENT_MODULE( FAkPresenceModule, AkPresence )
```

Expand code Copy full snippet (38 lines long)

UserIPresence.cpp

```
// Copyright Epic Games, Inc. All Rights Reserved.
IMPLEMENT_MODULE( FIPresenceModule, IPresence )
```

Expand code Copy full snippet (38 lines long)

Walkthrough

1. Both users retrieve the default online services by calling `GetServices` with no parameters specified and access the Presence Interface.

   UserAkPresence.cpp and UserIPresence.cpp

   ```
   // Get the Online Services Presence Interface
   IOnlineServicesPtr OnlineServices = IOnlineServices::GetServices();
   IOnlinePresenceWeakPtr PresenceInterface = OnlineServices->GetPresenceInterface();
   ```

   Copy full snippet (2 lines long)

2. `QuickUpdate` initializes an `FUserPresence` struct named `Presence`. Notice that we are using two of the aforementioned enumerations provided by the Presence Interface: `EUserPresenceStatus` and `EUserPresenceJoinability`.

   UserAkPresence.cpp

   ```
   // Create the presence update
   TPtrCastRef<IOnlinePresence> Presence = MakePtrCastRef<IOnlinePresence>();
   Presence->SetAccountId(QuickUserId);
   Presence->SetStatus(EUserPresenceStatus::Online);
   Presence->SetJoinability(EUserPresenceJoinability::Public);
   Presence->SetRichPresenceString(TEXT("Exploring the Great Citadel"));
   Presence->SetProperties(MakeMap(TEXT("whereami_class"), TEXT("whereami_class_miscellaneous"));
   ```

   Copy full snippet (6 lines long)

3. `QuickUpdate` initializes an `FUpdatePresence::Params` struct named `Params` with the parameters that will be passed to `UpdatePresence`.

   UserAkPresence.cpp

   ```
   // Create the update presence parameters
   FUpdatePresence::Params Params;
   Params.LocalUserId = QuickUserId;
   Params.Presence = Presence;
   ```

   Copy full snippet (3 lines long)

4. `QuickUpdate` calls `UpdatePresence` and processes the result with an `OnComplete` callback.

   UserAkPresence.cpp

   ```
   // Update presence
   OnlineServicesPtr->GetPresenceInterface()->UpdatePresence(MoveTemp(Params))
   .OnComplete([QuickUpdate](const TVariant<TUpdatePresenceResult, TOnlineResultInfo>& Result)
   {
   // Check for success
   if (Result.IsOnlineResult())
   {
   // Log success
   UE_LOG(LogTemp, Warning, TEXT("Successfully updated presence for user %s"), *Result.GetOnlineResult().AccountId.ToString());
   }
   else
   {
   // Log failure
   UE_LOG(LogTemp, Warning, TEXT("Failed to update presence for user %s: %s"), *Result.GetOnlineResult().AccountId.ToString(), *Result.GetOnlineResult().ErrorMessage.ToString());
   }
   });
   ```

   Expand code Copy full snippet (15 lines long)

5. `QuickUpdate` queries the presence of `QuickUser`. Inside the querier `OnComplete` callback, `QuickUpdate` first checks to ensure `QueryPresence` returned an "OK" status. If it did, then `QuickUpdate` is safe to retrieve the presence of `QuickUser`, and process the result or error of `GetPresence` accordingly:

   UserIPresence.cpp

   ```
   // Query presence
   OnlineServicesPtr->GetPresenceInterface()->QueryPresence(MoveTemp(QueryParams))
   .OnComplete([QuickUpdate](const TVariant<TQueryPresenceResult, TOnlineResultInfo>& Result)
   {
   // Check for success
   if (Result.IsOnlineResult())
   {
   // Log success
   UE_LOG(LogTemp, Warning, TEXT("Successfully queried presence for user %s"), *Result.GetOnlineResult().AccountId.ToString());
   // Get the presence
   TOptional<TSharedRef<IOnlinePresence::FPresence>> QuickUserPresence = QuickUpdate->GetPresenceInterface()->GetCachedPresence(QuickUpdate->QuickUserId);
   if (QuickUserPresence.IsSet())
   {
   // Log success
   UE_LOG(LogTemp, Warning, TEXT("Successfully retrieved presence for user %s"), *QuickUpdate->QuickUserId.ToString());
   // Log the presence
   UE_LOG(LogTemp, Warning, TEXT("Presence for user %s: %s"), *QuickUpdate->QuickUserId.ToString(), *QuickUserPresence.GetValue()->ToDebugString());
   }
   else
   {
   // Log failure
   UE_LOG(LogTemp, Warning, TEXT("Failed to retrieve presence for user %s"), *QuickUpdate->QuickUserId.ToString());
   }
   }
   else
   {
   // Log failure
   UE_LOG(LogTemp, Warning, TEXT("Failed to query presence for user %s: %s"), *Result.GetOnlineResult().AccountId.ToString(), *Result.GetOnlineResult().ErrorMessage.ToString());
   }
   });
   ```

   Expand code Copy full snippet (33 lines long)

If all function calls return without error, `QuickUpdate` now sees the updated status of `QuickUser`, and `QuickUpdate` can choose to make decisions based on this status. For example, `QuickUpdate` could access the `GetPresenceResult` to see `QuickUpdate` is online and their joinability status is public. Upon setting this status `QuickUpdate` could decide to join `QuickUpdate` and "Explore the Great Citadel" together.

Converting Code from Online Subsystem

The Online Services plugins are an updated version of the Online Subsystem plugins and will exist alongside one another for the foreseable future. The API functionality of the Online Services Presence Interface maps approximately one-to-one with the API functionality of the Online Subsystem Presence Interface. A few caveats include:

- `SetPresence` was renamed to `UpdatePresence`, to better represent the function's asynchronicity.
- `UpdatePresence` and `QueryPresence` are no longer overloaded.
- We recommend using their renamed functions `PartialUpdatePresence` and `BatchQueryPresence` instead.
  - The overloads for `UpdatePresence` and `QueryPresence` were renamed to `PartialUpdatePresence` and `BatchQueryPresence`, respectively.
- `QueryPresence` was given the `bLiveIfanyChanges` parameter.
- This adds a specific user to the `OnPresenceUpdated` event.
  - The parameter is set to true by default.

More Information

Header File

Consult the `Presence.h` header file directly for more information as needed. The Presence Interface header file `Presence.h` is located in the directory:

```
Engine/Plugins/Online/OnlineServices/Source/OnlineServicesInterface/Public/Online
```

Copy full snippet (1 line long)

For instructions on how to obtain the UE source code, refer to our documentation on Downloading Unreal Engine Source Code.

Function Parameters and Return Types

Refer to the Functions section of the Online Services Overview page for an explanation of function parameters and return types, including how to pass parameters and processing the results when functions return.

online beta programming presence multiplayer online services status